COMP 1005B/1405B
Winter 2022 - Tutorial 3

Objectives

- Practice using Python input/output operations
- Practice writing code using **conditional** statements

Expectations

To receive full grades for this tutorial, you must complete problems 1, 3-5. Problems 2 and 6 are for practice. If you do not have time to complete the problems, you should attempt them after the tutorial for your own benefit.

Submit

For this tutorial, your group will submit a single zip file called **YourGroupName_T3.zip** to the appropriate channel in Discord, where **YourGroupName** should be replaced by *your group name*. The zip file should contain the attendance record of your group members and the solutions to all the required problems. If any group members are absent during a tutorial session and do not participate in any group work during that entire week, your group must report it.

Ensure that your tutorial grade is entered on Brightspace at the end of each tutorial week. If you don't see your previous grades, contact the Tutorial TAs immediately. You have no more than 1 week to report a missing tutorial grade.

Warm-up

Problem 1

Write a program called **p1.py** that performs the function of a simple integer calculator. This program should ask the user for an operation (including the functionality for at least addition, subtraction, multiplication, and long division) and after the user has selected a *valid operation*, ask the user for the two operands (i.e., the integers used in the operation). If the user enters an *invalid operation*, the program should print out a message telling them so. Assume for now that the user always enters correct inputs for the operands (ints).

Note: for this tutorial, long division of integers requires that you provide both a quotient and a remainder. You can refer to https://docs.python.org/3/library/stdtypes.html or the lecture notes if you do not recall how to compute the remainder when you divide two integers. Refer to the sample outputs below, user inputs are highlighted.

Sample Output-1

```
(A)ddition
(S)ubtraction
(M)ultiplication
(D)ivision (Long)

Please select an operation from the list above: Delete
This program does not support the operation "Delete".
```

Sample Output-2

```
(A)ddition
(S)ubtraction
(M)ultiplication
(D)ivision (Long)

Please select an operation from the list above: M
Please provide the 1st integer: 3
Please provide the 2nd integer: 5
3 * 5 = 15
```

Sample Output-3

```
(A)ddition
(S)ubtraction
(M)ultiplication
(D)ivision (Long)
```

```
Please select an operation from the list above: D
Please provide the 1st integer: 16
Please provide the 2nd integer: 3
16 / 3 = 5 with remainder 1
```

Problem 2

Write a program called **p2.py** that asks the user for two integers representing the month [1, ..., 12] and the day [1, …, 31] respectively. Your program should then output, which of the four seasons ("Winter", "Spring", "Summer", or "Fall") the entered date is in.

You should use the following date ranges for each season:

1. **Spring**: March 20th to June 20th
2. **Summer**: June 21st to September 22nd
3. **Fall**: September 23rd to December 20th
4. **Winter**: December 21st to March 19th

Hint: You can use logical operators **'and'** / **'or'** to combine multiple Boolean expressions.

Example:

```
Enter month [1,12]: 3
Enter day [1,31]: 19
It's Winter!


Enter month [1,12]: 3
Enter day [1,31]: 21
It's Spring!
```

Note: Be sure to check that the input is valid. Some things to check may include:

- if it is a number
- if it is in the correct range of values

Tip: Your program may follow the following layout:

1. get user input
2. check if input is valid
3. determine season
4. output result

Hangman Game

Problem 3

Write a program **p3.py** that checks if user input is valid. This program should check that the input is a single letter.

Consider the following pseudocode to help you develop your solution:

```
x := user input

if x has length of 1 and is a letter
      print "valid"
else
      print "invalid"
```

Hint: make use of the built-in method `.isalpha()`.

This method returns true if all the characters are alphabet letters. You can read about this method here: https://www.w3schools.com/Python/ref_string_isalpha.asp

example using `.isalpha()`:

```
x = COMP
print(x.isalpha())
>>> true
```

Note: This question serves to practice if/else structure as well as boolean operators (remember the 'and' keyword)

Problem 4

Write a program called **p4.py** that prints out a classic hangman stick figure. The program should ask the user to enter a number from 1-6 and the corresponding hangman should be printed.

The value the user inputs corresponds to the number of incorrect guesses in a real hangman game and so the completeness of the hangman will correspond to the number of 'incorrect guesses' inputted by the user (e.g., if the user enters 1, then only the head of the hangman will be printed; full example below).

See several sample outputs below.

```
Enter a number from 1-6: 1
 O

Enter a number from 1-6: 2
 O
 |
```

```
Enter a number from 1-6: 3
 O
\|
 |

Enter a number from 1-6: 4
 O
\|/
 |

Enter a number from 1-6: 5
 O
\|/
 |
/

Enter a number from 1-6: 6
 O
\|/
 |
/ \
```

Feel free to use a more interesting / informative prompt than “Enter a number from 1-6”.

Hint: consider storing each piece of the hangman as a variable.

```
Head (1): O
Body (2): |
Arm 1 (3): \|
Arm 2 (4): \|/
Leg 1 (5): /
Leg 2 (6): / \
```

Problem 5

Modify your program from problem 4 so that the user input is checked to be a valid single digit (1-6) before printing the corresponding hangman. If the input is not valid, then instead of a hangman the following message should be printed “Invalid input: you must enter a single number from 1-6.” Name this program **p5.py.**

Example:

```
Enter a number from 1-6: 7
Invalid Input: you must enter a number from 1-6
```

Challenge: Instead of returning a generic error / invalid input message, try adding some specificity. i.e., if the inputted string is not a number, then the returned message should specify that the problem is that it is not a digit; if the inputted string is too long, then the returned message should specify that the problem is that it is too long / not a single character.

DNA Project

Problem 6

Write a multiple-choice question program called **p6.py**. This program should do the following:

1. Prompt the user with the following question:

```
What is the complement of the DNA strand AATG?
a) CCGT
b) GGCA
c) TTAC
d) GTAA
```

   Note the formatting of the question (i.e., the line breaks). Also there should be a line break at the end of the question.

2. The user will then input an answer. The input should be validated. Valid answers are single characters ‘a’, ‘b’, ‘c’, or ‘d’ - case insensitive (i.e., ‘A’ is also valid).

3. The program should then output whether or not the user’s answer is correct. The correct answer is ‘c’ or ‘C’. If the user guesses incorrectly, the correct answer should be printed, along with stating that the answer is incorrect.

Examples:

```
What is the complement of the DNA strand AATG?
a) CCGT
b) GGCA
c) TTAC
d) GTAA
C
Correct!
```

What is the complement of the DNA strand AATG?
a) CCGT
b) GGCA
c) TTAC
d) GTAA
a
Incorrect. The correct answer is: c) TTAC